

ChatBot for MIT Counselling Process

A project report submitted

to

MANIPAL ACADEMY OF HIGHER EDUCATION

For Partial Fulfillment of the Requirement for the

Award of the Degree

of

Bachelor of Technology

in

Computer and Communication Engineering

by

Ankit Agrawal

180953278

Under the guidance of

Dr. Gririja Attigeri

Assistant Professor(SI. Gr.)

Department of I & CT

Manipal Institute of Technology

Manipal, India



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL

(A constituent unit of MAHE, Manipal)

June 2022

I dedicate my thesis to my friends and family.

DECLARATION

I hereby declare that this project work entitled **ChatBot for MIT Counselling Process** is original and has been carried out by me in the Department of Information and Communication Technology of Manipal Institute of Technology, Manipal, under the guidance of **Dr. Girija Attigeri, Assistant Professor(SI. Gr.)**, Department of Information and Communication Technology, M. I. T., Manipal. No part of this work has been submitted for the award of a degree or diploma either to this University or to any other Universities.

Place: Manipal

Date :20-06-22



Ankit Agrawal



CERTIFICATE

This is to certify that this project entitled **ChatBot for MIT Counselling Process** is a bonafide project work done by **Mr. Ankit Agrawal (Reg.No.:180953278)** at Manipal Institute of Technology, Manipal, independently under my guidance and supervision for the award of the Degree of Bachelor of Technology in Information Technology.

Dr.Girija Attigeri

Assistant Professor(SI. Gr.)

Department of I & CT

Manipal Institute of Technology

Manipal, India

Dr.Smitha N. Pai

Professor & Head

Department of I & CT

Manipal Institute of Technology

Manipal, India

ACKNOWLEDGEMENTS

I wish to express my heartfelt gratitude to the all the people who have played a crucial role in the research for this project, without their active cooperation the preparation of this project could not have been completed within the specified time limit.

My deepest thanks to my Project Guide Dr. Girija Attigeri, Assistant Professor (Sl. Gr.), Department of Information and Communication Technology, Manipal Institute of Technology and Dr. Sucheta V Kolekar, Associate Professor, Department of Information and Communication Technology, Manipal Institute of Technology for their guidance and their mentor ship throughout the project.

I would also like to use this opportunity to thank my family and friends who encouraged me over the course of the project.

ABSTRACT

A chatbot is an artificially intelligent entity which can interact with humans and can mimic a conversation. They work on the principle of Artificial Intelligence (AI) and Natural Language Processing (NLP) to provide the answers to the user's queries. Chatbots will be very helpful for query resolution during the admission process of the college because it will provide official and uniform information and can be accessed 24x7. Therefore, the aim of the project is to design a system which will capture the user's query and will use NLP approach and semantic analysis to provide answers from the data repository.

In this project, five chatbot models were created using neural networks, TF-IDF vectorization, and pattern matching. In neural network-related models, a neural network is created and then trained with the dataset; when a query is taken as input, the neural network tries to identify the tag of that query and returns the corresponding response. In the TF-IDF-related model, cosine similarity is calculated between every sentence in the repository and the user's query; the sentence with the highest similarity is given as the response. In pattern matching model, AIML is used to create heuristic rules for pattern matching and the user's query is matched to any of the patterns.

From the project, it was observed that neural network-related models had better accuracy than TF-IDF and pattern matching models. It can also be observed that the chatbot with Sequential modeling was the most accurate model because it prevented overfitting. The pre-processing of data also had an impact on the performance of the model. Using different stemming algorithms affected the performance of the chatbot. Furthermore, a chatbot having any kind of optimizer yielded better results. Moreover, the type of activation functions and the layer to which it was applied also impacted the output of the model. Lastly, it was also observed that one needs a deep understanding of pattern matching to create a chatbot with acceptable results.

[**Computing Methodologies**]: Artificial Intelligence- Natural Language
Processing, Lexical semantics

Contents

Acknowledgements	iv
Abstract	v
List of Tables	ix
List of Figures	x
Abbreviations	x
1 Introduction	2
2 Literature Survey	7
3 Problem Formation	11
3.1 Problem Statement	11
3.2 Objectives	12
4 Methodology	13
4.1 Preprocessing	13
4.2 Neural Network	19
4.3 Design of Chatbots	21
4.3.1 Smart Bot: Neural Network using TensorFlow (TfLearn)	21
4.3.2 Sam: Neural Network using PyTorch	24
4.3.3 Big Mouth: Using TF-IDF Vectorization	25
4.3.4 Hercules: Using Sequential Modeling	28

4.3.5	ALICE:Using AIML	31
5	Result Analysis	35
5.1	Confusion Matrices and Accuracy based on sample test dataset	35
5.1.1	Smart Bot	35
5.1.2	Sam	36
5.1.3	Big Mouth	36
5.1.4	Hercules	37
5.1.5	ALICE	37
5.2	Query Analysis on Chatbots	38
5.3	Conversational Analysis of Chatbots	50
5.3.1	Smart Bot	50
5.3.2	Sam	52
5.3.3	Big Mouth	54
5.3.4	Hercules	55
5.3.5	ALICE	57
6	Conclusion and Future Scope	60
	Appendices	61
	References	61
	ProjectDetail	66

List of Tables

5.1	Confusion matrix for SmartBot	36
5.2	Confusion matrix for Sam	36
5.3	Confusion matrix for Big Mouth	36
5.4	Confusion matrix for Hercules	37
5.5	Confusion matrix for ALICE	37
5.6	Accuracy of each chatbot	38
5.7	Queries.	38
5.8	percentages	50
6.1	Project Detail	67

List of Figures

4.1	Basic flow of a chatbot	14
4.2	Structure of a Neural Network	19

ABBREVIATIONS

NLP	:	Natural Language Processing
XML	:	Extensible Markup Language
AIML	:	Artificial Intelligence Markup Language
IR	:	Information Retrieval
TF-IDF	:	Term Frequency and Inverse Document Frequency
AI	:	Artificial Intelligence
RNN	:	Recurrent Neural Network
LSTM	:	Long short-term memory
ALICE	:	Artificial Linguistic Internet Computer Entity
MU OET	:	Manipal University Online Entrance Test
MIT	:	Manipal Institute of Technology

Chapter 1

Introduction

A chatbot is an artificially intelligent entity which can interact with humans and can mimic a conversation. The input to the chatbot could be text-based or spoken (voice-based queries). Chatbots are majorly used for information retrieval. It can run on a local computer or on mobile phones, though most of the time chatbots are accessed through the web browser. Mainly a chatbot works by a user asking some question or query regarding a specific topic. They work on the principle of Artificial Intelligence(AI) and Natural Language Processing(NLP) to provide the answers to the user's queries and a predefined knowledge base helps develop a response to the query [1].

There are three major types of chatbots, namely Rule-based, Retrieval-based model, Generative-based model. In the Rule-based Model, the bot responds to queries using a series of pre-programmed set of rules. This kind of chatbots can answer simple and a limited set of queries. Retrieval-based Models selects an appropriate response from a set of pre-defined responses using limiting conditions(heuristic). The bot created is capable to understanding the entire conversation and respond based on the context of the conversation. Lastly, Generative-based models generate response from previous and current experience. This type of chatbots are extremely sophisticated, requiring complex computational models and a vast quantity data to train. [2].

Types of Neural Network Language Models are:

- **Recurrent Neural Network(RNN):** RNNs are meant to take text sequences as inputs and return textual content sequences as outputs, or both. They're called recurrent because the hidden layers of the network feature a loop in which the output and cell state from each step are inputted at the following step. The ability of an RNN to memorise the results of past computations and utilise those records in the present computation is its fundamental strength.
- **Long Short Term Memory(LSTM):** Long Short Term Memory Networks (LSTMNs) are a special sort of RNN that can learn long-term dependencies. [3]

Natural Language Processing(NLP) is a subfield of Computer Science that uses computational methods to learn, comprehend and produce text in human language. Syntactic analysis and semantic analysis are the two primary techniques that lead to the understanding of natural language. Syntactic means the placement of words in a sentence such that the sentence is grammatically(or syntactically) correct. In addition, syntactic analysis transforms a set of words into structures that depict the relation between words. On the contrary, Semantic analysis is concerned with the meaning of each word and sentence. Semantic analysis of a sentence or a phrase captures its actual meaning and analyses the logical structure of sentences to uncover word similarities. [4]. The significant applications of NLP include Classification and Clustering, Data Mining, translation of texts and Semantic extraction.

There are three major types of queries:

- **Simple Query:** A simple query consists of a single and unconstrained query desire(desired output) and has a single unconstrained query input.

For example, in the query "What is the capital of USA?", the desired output of the query(i.e. Capital) is explicit, single and not bound to any constraint. The input to the query(i.e. USA) is also single and unconstrained.

- **Complex Query:** A complex query consists of a single query-desire, which can be either constrained or unconstrained and the input to the query is multiple and explicit but it can be constrained or unconstrained. For example, in the query "What was the capital of USA during World War II?", the query has multiple inputs(i.e. USA and World War II) and the desired output is single, unconstrained and implicit(i.e. Capital).
- **Compound Queries:** A compound query is a query which has a conjunction or a disjunction operators which connects two simple or complex queries. For example, "What are the capitals of USA and Germany?" , this is a compound query because it has "and" in it.

Data preprocessing is the most essential step when it comes to Machine Learning. Machines don't understand words so the data needs to be converted to machine understandable language; that is; numbers. Data preprocessing includes the following procedures:

- **Letter Case:** The complete corpus of predefined responses or user queries must be converted into a single case, preferably lower case. This helps the algorithm maintain uniformity by ensuring that it does not treat the same words in different cases as separate words [2].
- **Tokenization:** Tokenization is the process of transforming normal text-based strings into a list of tokens that can be further processed. Word tokenizer and Sentence tokenizer are the two processes used to make a list of words and sentences respectively [2].

- **Removing Noise:** It involves the removal of everything other than alphanumeric characters [2].
- **Removing Stop Words:** Stop words are the words that are of negligible value in determining the response to the user's query. Such words have to be removed from the list of tokens because they do not provide any useful information [2].
- **Stemming:** Stemming is the process of converting words to their root form. For example, the root form of all the words "writes", "writing", "written" is the single word "write" [2].
- **Lemmatization:** Lemmatization is the process of converting every word in a sentence to its canonical form. For example, the canonical form of "did" is "do" [5].
- **Bag of Words:** creation of Bag of Words is the process of converting wall text into a list of zeroes and ones. This list indicated whether a word is present in the sentence or not. Bag of Words is fed as input to the neural network because it can not understand human language.

AIML stands for Artificial Intelligence Markup Language and it is an XML base markup language meant to create artificial intelligent applications. In 2021, Md Mabruur Husan Dihyat et al . [6] wrote that AIML uses pattern matching techniques to formulate answers from queries. The basic unit of AIML script is called category tag which is formed by user input patterns and chatbot responses according to the input. Inside each category, the question is stored in the *< pattern >* tag while the corresponding answer to the question is stored in the *< template >* tag. The pattern consists of words, spaces and wildcard symbols such as: $\wedge$ and $*$. Wildcards symbols are used to replace strings in AIML.

Manipal Institute of Technology has an elaborate admission process including a special entrance exam Manipal University Online Entrance Test(MU OET), counselling and stream selection process. Students and Parents have a lot of queries regarding the entrance process which are clarified either on calls or by visiting the university. Due to the shear volume of queries,and limited personnel the query resolution is problematic. There might be some miscommunication of information, or many a times officials get busy on other calls which prevents any other student from getting their queries resolved. Not only this, but also students need to go through the whole website in order to get a specific information which can be tedious. [7]

A chatbot might be a good solution for answer the student's queries regarding the admission process. It will provide 24x7 assistance and the information provided will be uniform and clear.

Chapter 2

Literature Survey

In 2017, Vibhor Sharma et al. [8] authors wrote that **ELIZA** was created by a German Computer Scientist , Joseph Weizenbaum in 1966. It is considered to be the first chatbot in computer history. Eliza used "pattern matching" and substitution methodologies to simulate conversations. In 2020, Eleni Adamopoulou et al. [9] authors that ELIZA responds like a psychotherapist by returning the user's query in an interrogative form. The downside of ELIZA is that its knowledge is limited and therefore it can only discuss a limited range of topics. Furthermore, ELIZA is unable to maintain long conversations and is unable to learn context from the conversation.

In 2020, Eleni Adamopoulou et al. [9] authors wrote about a chatbot called **PARRY** which was created in 1971. PARRY is considered to be more evolved than ELIZA since it had a "personality" and a more effective control structure. In 2012, Sandeep A Thorat et al. [10] authors wrote that ELIZA chatbot system also has language comprehension capabilities and can has variables like mistrust, anger and fear.

In 2006, Luke Fryer et al. [11] authors wrote about **Jabberwacky**, which was written in CleverScript, an Artificial Intelligence tool. In 2020, Eleni Adamopoulou et al. [9] authors wrote Jabberwacky was created in 1988 and

it used contextual pattern matching algorithms to answer queries based on previous discussions. In 2020, Verma Shivang et al. [12] authors wrote that Jabberwacky's main goal was to transition from a text-based system to a fully voice-driven system.

In 2020, Shivang Verma et al. [12] authors wrote about Artificial Linguistic Internet Computer Entity (**ALICE**), which is Natural Language Processing chatbot which uses heuristical pattern matching algorithms to conduct conversations. created by Dr. Richard Wallace. It was first released on November 23,1995, and was redone in the JAVA programming language in 1998. ALICE was written using Artificial Intelligence Markup Language(AIML); which is an XML based schema for writing heuristic conversational rules.In 2020, Eleni Adamopoulou et al. [9] authors wrote that ALICE completely relied on pattern matching algorithms without actually recognizing the context of the entire conversation. Also, ALICE lacks intelligent traits and therefore it can not generate human-like responses that expresses emotions and attributes.

In 2020, Shivang Verma et al. [12] authors wrote about **Mitsuku**, which is an intelligent chatbot which was created by Steve Worswick using AIML. In 2020 Santosh Maher et al. [3] authors wrote that Mitsuku was created for a general type conversation and it interacts with the user using the rules which are written in AIML. Mitsuku can also be integrated with social media platforms such as telegram or Twitter.The chatbot is hosted at Pandorobot and employs NLP with heuristic patterns. Mitsuku is capable of retaining large amounts of conversational history and utilizing it in future conversations.

In 2020, Shivang Verma et al. [12] authors wrote about **Siri**; a virtual assistant developed by Apple launched in 2010. Siri uses a natural language interface that enables it to take actions, make recommendations and perform specific actions in response to voice queries. With users' usage, Siri is capable of adapting to the user's language usages and searches. Siri has many features

including handling device settings, scheduling events and searching queries on the internet. In 2020, Eleni Adamopoulou et al. [9] authors wrote about Siri's weaknesses. Siri's main weakness is the it is dependant on the internet to function. Siri can understand many languages, but there are many languages which are not supported; while the navigational instructions are only available in English. Furthermore, Siri struggles to understand strong accents and commands in the presence of external noise.

In 2019, Alaa A Qaffas [13] wrote about IBM's **Watson**. Watson is a question and answer unit that can provide answers to questions asked in natural language. Watson uses NLP and machine learning algorithms to extract insights from previous conversations. The downside of Watson is that he only supports English. In 2020, Eleni Adamopoulou et al. [9] authors wrote that Watson was created in 2011 and later "Watson Health" was created which helped doctor to diagnose diseases.

In 2020, Eleni Adamopoulou et al. [9] authors wrote about **Google Assistant**; which consists of next generation of **Google Now**. Google Now was created by Google in 2012 and it gave responses based on the users' preference and location. Google Assistant is a deeper artificial intelligence and has a friendlier user interface. The main disadvantages of Google Assistant is that it has no personality and it violates the user's privacy because it is irectly linked to their google accounts.

In 2020, Eleni Adamopoulou et al. [9] authors wrote about **Cortana**; a digital assistant which was developed by Microsoft in 2014. It understands voice instructions and can identify time and location, send emails, create reminders, and manage lists. Cortana has a significant flaw in that it can run a software that installs malware.

In 2016, Martin Abadi et al. [14] authors wrote that TensorFlow is a pro-

programming language for expressing and executing machine learning algorithms. With few or no adjustments, TensorFlow computations can be conducted on a wide range of heterogeneous systems, ranging from mobile devices like phones and tablets to large-scale distributed systems. The framework is extensible, and it may be used to define a variety of algorithms, such as deep neural networks and inference approaches.

In 2018, Shahzad Qaiser et al. [15] authors wrote that Term Frequency and Inverse Document Frequency (TF-IDF) is a numerical statistic that illustrates the relevance of keywords to certain documents, or in other words.

In 2015, Imran Ahmed et al. [16] authors wrote that Artificial Intelligence Markup Language (AIML) has a syntax similar to Extensible Markup Language (XML) and is used for pattern matching algorithms.

In 2018, Ruby Rani et al. [17] authors wrote that in IR systems, stop words are words that have little or no semantic importance. The removal of such common words can lead to more effective corpus indexing and boost the performance of an IR system.

In 2021, Dan Ofer et al. [18] authors wrote that the process of splitting text into atomic units of information in a selected language representation (called tokens), is known as tokenization. Although some approaches employ individual letters, most English NLP models use words as tokens. Individual-character tokens provide more versatility, particularly for out-of-vocabulary or misspelt words, as well as languages with a lack of unambiguous word division.

Chapter 3

Problem Formation

3.1 Problem Statement

Engineering colleges follow an online admission process along with counselling for providing information regarding various courses. Students and Parents have a lot of queries regarding the entrance process which are clarified on calls or by visiting the university. There might be some miscommunication of information, or many a times officials get busy on other calls which prevents any other student from getting their queries resolved.

Engineering colleges follow an online admission process which constitutes a counseling process and stream selection. During this phase, the students and parents have various kinds of questions, such as:

- Queries regarding the college
- Queries regarding the branch they are about to select
- Queries regarding the students' options after college
- Queries regarding the placements in the final year
- Queries related to the various branches and the difference between them along with questions related to the relatively new branches.

All the queries can be clarified by visiting the college or over a phone call with the officials. Due to the sheer volume of queries it might be difficult for the officials to answer all the queries properly which may lead to miscommunication of information, and sometimes they might be busy on some other call which prevents the student from getting the information they need.

Students may have to rely on online platforms such as Quora or Telegram groups for getting information and getting their queries resolved. These sources of information are not reliable because the information is not provided by a university official. Furthermore, students might need to navigate through the entire college website to find a particular piece of information which can be tedious.

3.2 Objectives

The objectives of the project are:

- Design and development of system to capture and analyse user's query.
- Design the NLP based approach for features and semantic extraction.
- Design of an effective approach to model and identify answer from domain data repository.

Chapter 4

Methodology

This chapter describes the work done in the project. This chapter is divided into 3 sections. In the first section, different pre-processing steps are explained. The second section defines neural networks and its parameters. Lastly, the third section consists of the five chatbots models created.

Figure 4.1 depicts the basic process while developing a chatbot.

4.1 Preprocessing

In 2019, Tohari Ahmad et al. [19] authors wrote that the raw data is pre-processed so that it can be converted into a format that is easier and more effective to use for further processing steps. In 2019, Biveeken Vijayakumar et al. [20] authors wrote that the purpose of data preprocessing is to normalize the raw data in the dataset and to reduce the number of features in the feature set. This leads to decrease in complexity of fitting the data to each classification model.

The pre-processing steps are explained below:

- **Converting to Lowercase:** In 2019, Biveeken Vijayakumar et al. [20] authors wrote that the raw text is changed to lowercase to avoid numerous variants of the same word, and all the words, regardless of their

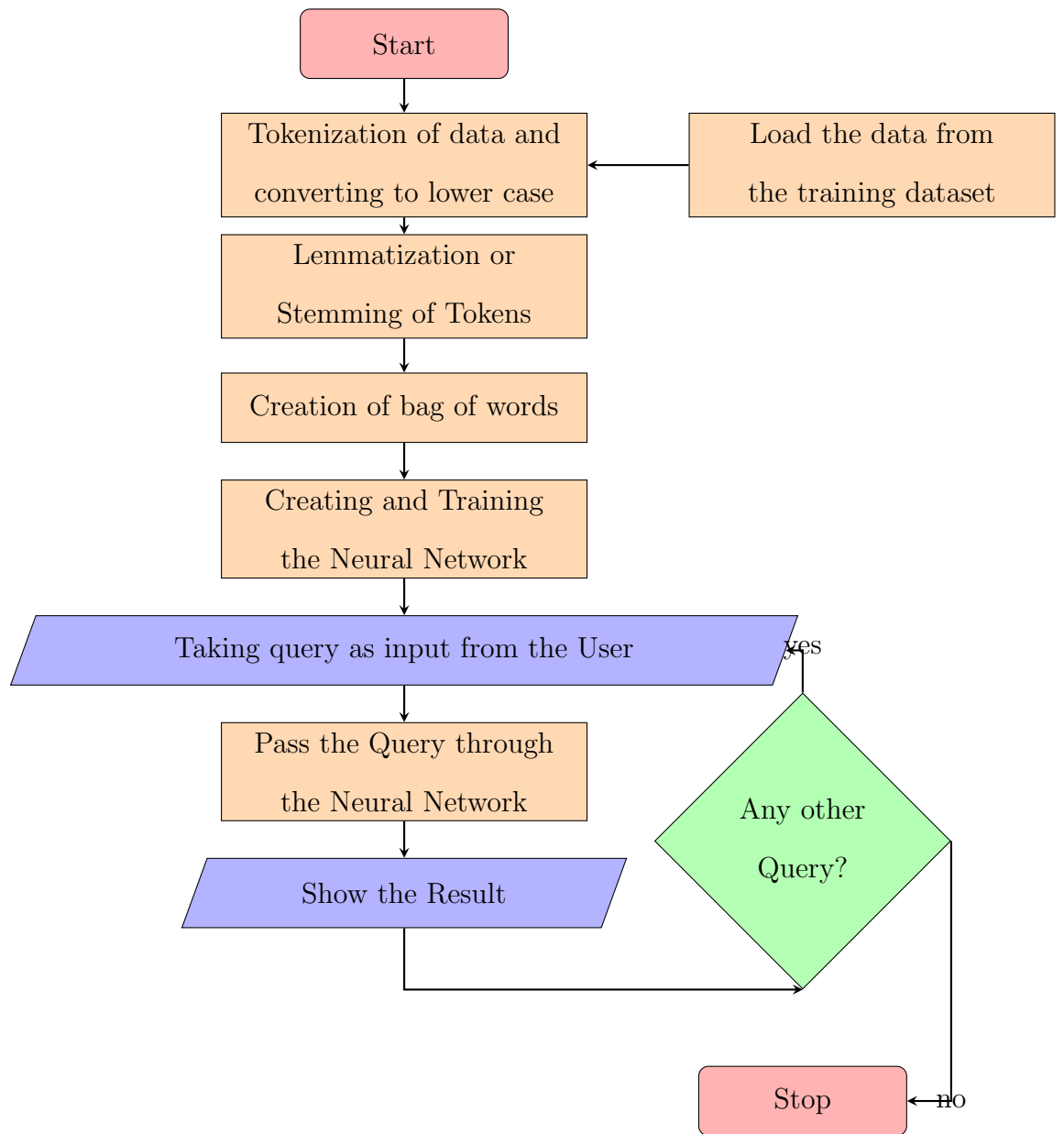


Figure 4.1: Basic flow of a chatbot

casing, are standardized/normalized to lowercase so that they can be counted together. Algorithm 1 is a pseudo code for converting the text to lowercase.

Algorithm 1 Converting Raw Text to Lower Case

```

data ← rawText
outputString ← ""
while ch ∈ data do
    if  $\text{ascii}(ch) \geq 65$  &  $\text{ascii}(ch) \leq 90$  then
        outputString ←  $\text{char}(\text{ascii}(ch) + 32)$ 
    else
        outputString ← outputString + ch
    end if
end while
return outputString

```

- **Tokenization:** In 2014, Tanu Verma et al. [21] authors defined tokenization as the process to dividing a text stream into meaningful elements called tokens. Tokens can be words, sentences or any other part of the sentence. Algorithm 2 contains the code for the two types of tokenization. The intent behind the use of tokenization is identification of meaningful keywords.

Algorithm 2 Algorithm for Tokenization

```

Data= Sentence input
nltk.sent_tokenize(Data)
nltk.word_tokenize(Data)
Output=Tokenized Words and Sentences in the form of lists

```

- **Bag Of Words:** Neural networks can not understand words and hence require numbers as input. Therefore, Bag of Words model is used to convert words into machine recognizable vectors of numbers. There are two types of Bag of Words; one is a list of zeros and ones which signifies if the word is present in the sentence or not. The other kind keeps count of the number of occurrences of most frequently used words. The size

of the Bag of Words is the number of unique root words. The Bag of Words is represented as a list of 0s and 1s where each position in the list represents if a word exists or not in the sentence. Algorithm 3 is the pseudo code for creating bag of words.

sentence=hello, how are you?

words=["are", "bye", "hello", "hi", "how", "i", "thank", "you"]

bag=[1,0,1,0,1,0,0,1]

Algorithm 3 Bag of Words Algorithm

```

dataset=dataset with lowerCase alphabets
Output=bag of words
word2count=empty dictionary
while  $data \in dataset$  do
     $words \leftarrow tokenized\ data$ 
    while  $word \in words$  do
        if  $word \notin word2count.keys()$  then
             $word2count[word] \leftarrow 1$ 
        else
             $word2count[word] \leftarrow word2count[word] + 1$ 
        end if
    end while
end while
 $freqWords \leftarrow largest\ value\ in\ word2count$ 
X empty list
while  $data \in dataset$  do
    vector=empty list
    while  $word\ in\ freqWords$  do
        if  $word \in tokenize(data)$  then
            append 1 to vector
        else
            append 0 to vector
        end if
    end while
    append vector to X
end while
 $X \leftarrow np.array(X)$ 

```

- **Removing Stop Words:** In 2018 Jashanjot Kaur et al. [22] authors wrote that stopwords are the words that occur most frequently in a

document and contain very little information that is not usually relevant. For example, in English language, there are some words such as "a", "about", "above", "after", "again" and "against", all contain meagre information and thus they are called stopwords. Removing stopwords does not only reduces vector space, but also improves the performance of the model by increasing accuracy and reducing the training time and number of calculations. Algorithm 4 is the pseudo code for removing stop words.

Algorithm 4 Algorithm to remove StopWords

```

exampleSent  $\leftarrow$  sampleSentence
stopWords  $\leftarrow$  set(stopwords.words('english'))
wordTokens  $\leftarrow$  wordtokenize(exampleSent)
filteredSentence  $\leftarrow$  [w for w in wordTokens if not w.lower() in stopWords]
filteredSentence  $\leftarrow$  []
while w  $\in$  wordTokens do
    if w  $\notin$  stopWords then
        filteredSentence.append(w)
    end if
end while

```

- **Stemming:** In 2018, Ammar Ismael Kadhimi et al. [23] authors wrote that stemming is the process of removing prefixes and suffixes (affixes) of words. When several types of features are stemmed into a single feature, it reduces the amount of features in the feature space and increase the performance of the classifier. In 2019, Biveeken Vijayakumar et al. [20] authors wrote that stemming is a technique used for combining multiple versions of a word that share the same stem into a single root form. For example, the words "likes", "like", "likely", "liked" and "liking" will all be stemmed to their root form "like". Consolidating all versions of a term with the same root form guarantees that the root form's frequency is assessed collectively, increasing the likelihood of the root form appearing. In 2010, V. Srividhya et al. [24] authors wrote that the hypothesis behind stemming is that words with the same stem or word root are thought to

describe the same or similar concepts in text, hence stems can be used to conflate words.

In 2020, Divya Khyani et al. [25] authors wrote that stemming is used to shorten the lookup time and normalize the sentences for a better understanding. There are various methods to implement stemming, a few are mentioned below:

- **Porter Stemmer Algorithm:** It is one of the most popular stemming methods proposed in 1980. This stemming Algorithm is used to remove and replace suffixes of English words to make words simpler and efficient. This stemmer is known for its speed and simplicity. For Example "racing" becomes "race". Algorithm 5 shows the use of Porter stemmer algorithm.

Algorithm 5 Porter Stemmer Algorithm

```
wordStemmer ← PorterStemmer()
wordStemmer.stem('racing')
Output = race
```

- **Lancaster Stemming Algorithm:**It is named after the university "Lancaster" as it was developed there. The stemmer is really fast, but the algorithm is really confusing when dealing with small words. For Example: "eats" becomes "eat". Algorithm 6 shows the use of Lancaster stemmer algorithm

Algorithm 6 Lancaster Stemming Algorithm

```
lancStemmer ← LancasterStemmer()
lancStemmer.stem('eats')
Output = eat
```

- **Lemmatization:** In 2019, Biveeken Vijayakumar et al. [20] authors wrote that Lemmatization and stemming are quite similar as they both entail combining several versions of a word into a single root form. When

reducing and aggregating words, lemmatization takes into account the context and lemma of each word. As a result, it makes no distinction between words with slightly different meanings. The words "went," "going," "gone," and "go" will all become "go," despite the fact that "went" has distinct characters than "go," which the stemming algorithms do not do. Algorithm 7 shows the use of lemmatization

Algorithm 7 Algorithm for Lemmatization

```
word=Sample word
Lemmatizer  $\leftarrow$  wordNetLemmatizer
Output=Lemmatizer.lemmatize(word)
```

4.2 Neural Network

In 2019, Aarsh Trivedi et al. [26] authors wrote that the neural network framework is based on the biological neural network that formed inside the human brain. An input layer, one or more hidden layers, and an output layer make up an artificial neural network. Each artificial neuron in each layer is connected to the neuron in the next layer and has a weight assigned to it.

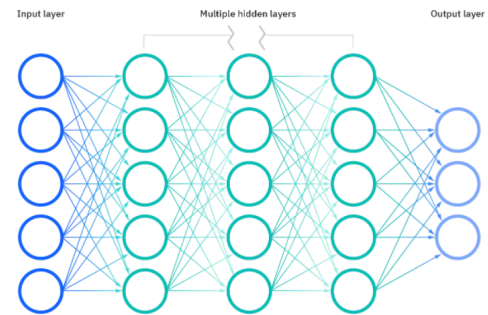


Figure 4.2: Structure of a Neural Network

A Feed Forward Neural Network is one of the most fundamental type of neural network because it conveys information in only one direction. On the contrary to feed forward neural networks, Long Short-Term Memory (LSTM) uses recurrent neural network, where the information flow is non-linear. While dealing with sequential data or data with a temporal link, LSTMs are favoured. However, LSTMs have their own disadvantages: they are comparatively slow

and require a large high-quality dataset in order to get acceptable results.

In 2015, Zachary C. Lipton et al. [27] authors wrote that the LSTM model looks like a typical recurrent neural network with a hidden layer, but each regular node is replaced by a memory cell in the hidden layer. A memory cell is a composite unit made up of smaller nodes connected in a specific configuration. A memory cell is a composite unit made up of smaller nodes linked together in a certain pattern. These ties help the network perform better on timed tasks that require it to learn to measure accurate intervals between occurrences.

In 2018, Rezoana Bente Arif et al. [28] authors wrote that in Rectified Linear Unit (ReLu) activation function if the input is negative, it returns 0, but if the input is any positive value x , it returns x .

$$\begin{cases} 0 & ; x < 0 \\ x & ; x \geq 0 \end{cases} \quad (4.1)$$

In 2018, Rezoana Bente Arif et al. [28] authors wrote that softmax activation function is generally used for classification purposes and it can be applied to the last layer/output layer of a neural network. This activation function is superior to other functions for classification purposes because it compresses the output of each segment between 0 and 1.

In 2010, Shivam Sinha et al. [29] authors wrote that epoch is a hyperparameter that defines the number times that the learning algorithm will run over the entire training dataset. It should have optimal value in the network because if it is less then it will not be able to train the network effectively. On the other hand, if the value of epoch is large, then it may cause the problem of overfitting where the model will give high errors when unfamiliar test data presented to the network.

4.3 Design of Chatbots

There are many ways of creating a chatbot and all of them will have different performances. Even if same query is fed to all the chatbots, the responses given by them might be different. In this section, the following five chatbots are explained:

- Smart Bot: In this chatbot, neural network is created using TensorFlow(TfLearn)
- Sam: In this chatbot, neural network is created using PyTorch
- Big Mouth: This chatbot is created using TF-IDF Vectorization
- Hercules: This chatbot is created using Sequential Modeling
- ALICE: This chatbot is created using AIML

4.3.1 Smart Bot: Neural Network using TensorFlow (TfLearn)

As the name suggests, TensorFlow runs computations based on tensors. In terms of machine learning, a tensor is a generalization of vectors and matrices, which are represented as an n-dimensional array of a base datatype. All elements of a tensor always have the same data type.

The explanation of Algorithm 8 is given the following paragraphs. The data is stored in *intents.json* file and it contains a list of intents. Each intent or class has a tag, a pattern, and a response. The "tag" defines the intent or class. The "pattern" is a list of possible questions of the corresponding class. The "response" is a list of possible answers to the questions of that "tag". The chatbot will take the message from the user and identify the "tag" of the message and will give the corresponding response.

Every "pattern" of every "intent" is tokenized using `nltk.word_tokenize()` and is appended to "words" list. All the tags are stored in the "labels" list. Now, "words" is a list that contains all the words in the database. Every word in "words" is converted to lowercase using the `lower()` function. Now, all the words in the "words" list are stemmed using `LancasterStemmer().stem()` function and all the duplicate words are removed using `set(words)`. "words" and "labels" are then sorted. A bag of words as "bag" (empty list) is created, where the size of "bag" is the number of root words in the database. For every word in "words" list, if that word exist in the sentence, then 1 is appended to "bag", else 0 is appended to "bag".

A Deep Neural Network is created using TfLearn. The size of the input layer is equal to the size of "bag". The input to the neural network is "bag". Two fully connected hidden layers of eight neurons each are added to the network. Fully Connected layers means that all possible connections are present, wherein, every input of the input vector influences every output of the output vector. An output layer of size equal to the number of tags in the dataset is added to the network. Softmax activation function is applied to each neuron in the output layer. "n_epochs" is the number of times the model will see the same training data. In this model "n_epochs" is set to 1000. Softmax activation function converts the output to a list of probabilities, with each value denoting the probability that the sentence belongs to the corresponding tag and the sum of all probabilities is equal to 1. After the model is trained, the variables are stored in data.pickle file. The training dataset is passed through the model in the form of bag of words and the model is trained. When the user query is passed through the neural the network, the tag with the highest probability is chosen and the corresponding response is given to the user.

Algorithm 8 Algorithm for Chatbot using TensorFlow

data $\leftarrow$ load data from JSON file

If the model has already been trained, load the variables from the pickle file

Initialize lists *words*, *labels*, *docs_x*, *docs_y*

while *intent* $\in$ *data* **do**

while *pattern* $\in$ *intent* **do**

wrds $\leftarrow$ tokenize words in the pattern

 Append *wrds* to *words*

 Append *wrds* to *docs_x*

 Append tag of the intent to *docs_y*

end while

if *tag* $\notin$ *labels* **then**

 Append tag of the intent to *labels*

end if

end while

remove punctuations from "words"

Stemming and converting to lowercase of "words" and store in "words"

sort "words" and "labels"

Initialize lists: training, output

while *sentence* $\in$ *docs_x* **do**

Initialize "bag" (bag of words)

stem every word in "sentence" and store in "wrds"

while *word* $\in$ *words* **do**

if *word* $\in$ *wrds* **then**

 append 1 to *bag*

else

 append 0 to *bag*

end if

end while

 Append "bag" to "training"

output_{row}[*labels.index(docs_y[x])*] = 1

 Append "bag" to "training"

 Append *output_{row}* to *output*

end while

convert "training" and "output" to array and save the variables in pickle file

create a Deep Neural Network using tflearn. The size of the input layer is same as the size of the bag of words. Then add 2 hidden layers (fully connected) of 8 neurons each. The size of the output layer is equal to the number of tags

set "number of epochs" to 1000

save the model

4.3.2 Sam: Neural Network using PyTorch

The explanation of Algorithm 9 is given the following paragraphs. The data is stored in *intents.json* file and it contains a list of intents. Each intent or class has a tag, a pattern, and a response. The "tag" defines the intent or class. The "pattern" is a list of possible questions of the corresponding class. The "response" is a list of possible answers to the questions of that "tag". The chatbot will take the message from the user and identify the "tag" of the message and will give the corresponding response.

Preprocessing steps are applied on the data. Every question of every intent is tokenized using *nlk.word_tokenize()* and is appended to "all_words" list. Every unique tag is stored in "tags" list. Now, "all_words" is a list that contains all the tokenized words of the dataset and "tags" is a list that contains all the tags of the database. All the punctuation tokens are removed; every word in the dataset is converted to lower case using the *lower()* function and the words are stemmed using *PorterStemmer().stem()* function from nltk. "all_words" list is sorted using *sorted(all_words)* function and all the duplicate words are removed using *set(all_words)* function. "tags" list is also sorted. To create a bag of words, "bag" list is created. "bag" list is of length equal to the size of list "all_words" or the number of unique stemmed words in the database. "bag" list is initialized with 0. For every word in "sentence", use its corresponding "index" in "all_words" to set bag[index] to 1.

A Feed Forward Neural Network is created using *torch.nn.Module*, which is a base class for all neural network modules. A feed forward neural network is an artificial neural network in which the connections between nodes does not form a cycle. One linear input layer of size equal to the size of "bag" list is created. Two hidden linear layer having 8 neurons are created. One output layer of size equal to size of "tags" list is created. A ReLu activation function is defined. Training data is passed through the input layer, then

the activation function is applied. This data is fed to the hidden layer and then the activation function is applied whose result is fed to another hidden layer. Activation function is applied and this data is fed to the output layer. Learning rate is an important hyperparameter which determines how fast the neural network would converge to optimum value. In this model, the value of learning rate is 0.001. When the user query is passed through the neural network, the "tag" with the highest probability is chosen and the response is given to the user.

4.3.3 Big Mouth: Using TF-IDF Vectorization

The term "Term Frequency" (TF) is used to count how many times a term appears in a document. Let's say there are 5000 words in document "T1," and the word "alpha" appears 10 times. As a result, the term "alpha" frequency in document "T1" will be

$$\mathbf{TF} = t/s$$

where t is number of occurrences in a file and s is the total number of words in the document

$$\mathbf{TF} = 10/5000 = 0.002$$

The inverse document frequency gives less weight to frequently occurring words and more weight to infrequently occurring words. For example, if we have ten documents and the term "alpha" appears in five of them, we may calculate the inverse document frequency as

$$\mathbf{IDF} = \log(M/m)$$

where M is the total number of documents in the corpus and m is then number of documents

Algorithm 9 Algorithm for Chatbot using PyTorch

Data = Load JSON Data

Initialize lists tags, xy, all_{words}

while *intent* $\in$ *data* **do**

Tag of the intent is stored in "tag"

while *question* $\in$ *intent[question]* **do**

$xy \leftarrow xy + \text{tokenized sentence}$

end while

end while

Remove stop words

Apply stemming, remove all duplicates, sort it and add them to "tags"

Initialize lists X_{train}, y_{train}

while *pattern* $\in$ *xy* **do**

while *tag* $\in$ *xy* **do**

Create bag of words using "pattern" and "all_{words}" and store in "bag"

append "bag" to X_{train}

append tag of intent to label

append "label" to y_{train}

end while

end while

A Forward Neural Network is created using PyTorch with 2 hidden layers with Relu activation functions.

The output layer is of size equal to the number of tags in the database. Relu activation function is applied to the input layer and the hidden layers. The tag with the highest probability is chosen. Training loss is calculated and printed after every 100 epochs and then the final loss is computed.

query = input from user

X = query is tokenized and converted to bag of words

output = query is passed through the model

if *probability* $>$ 0.75 **then**

if *tag* $\in$ *tags* **then**

 Randomly print one of the responses in that tag

end if

else

 Print that the bot does not understand the query

end if

containing the required term

$$\text{IDF} = \log(10/5) = 0.301$$

Algorithm 10 Algorithm for TF-IDF Vectorization

Data= Query input from the user

Output= Response from chatbot

sentTokens $\leftarrow$ Sentence tokenized data from database

Append the query to *sentTokens*

tfidf $\leftarrow$ TF – IDF vectorized data with stop words removed

vals $\leftarrow$ Cosine Similarity between the user query(*tfidf*[–1]) and *tfidf*

reqTFIDF $\leftarrow$ Maximum Cosine Similarity in *vals*

if *reqTFIDF* > 0 **then**

return the corresponding response

else

return "I do not understand.."

end if

Algorithm 11 Algorithm for Greeting in TF-IDF Vector chatbot

Data=Query from the user

Output=Response from the Chatbot if the query is a greeting

greetingInput $\leftarrow$ List of greeting input words

greetingOutput $\leftarrow$ List of greeting output words

while *word* $\in$ *query* **do**

if *lowerCase(word)* $\in$ *greetingInput* **then**

return random greetingResponse

end if

end while

In 2020, Abhishek Jaglan et al. [30] authors wrote that textual data can not be employed in the model directly, instead it has to be converted to numerical vectors. This can be done by assigning a unique number to each word, and given data can be encoded with the length of vocabulary of known words. The Bag-of-Words model is a way of representing whether the words exists in the "sentence" or not regardless of their sequence of appearance.

Term Frequency-Inverse Document Frequency (TF-IDF) stores the component of resulting scores assigned to each word. The goal of TF-IDF vector is to

calculate the word frequency scores for the text that are more interesting (less common). Term Frequency is used to calculate the frequency of each word, whereas, Inverse Document Frequency down scales the score of frequently occurring words.

The explanation of Algorithm 12 is given the following paragraph. The corpus consists of full stop separated answers in the form of a text file. The corpus is loaded and is tokenized using *nltk.word_tokenize* and *nltk.sent_tokenize*. The tokens are lemmatized using *nltk.stem.WordNetLemmatizer().lemmatize()*. Words which are there in *string.punctuation* (set of punctuations) are removed. Input is taken from the user in form of a "query". Greeting is implemented using the pattern matching algorithm. If the query contains any words from GREETING_INPUT (list of predefined greeting inputs), then the chatbot will return a random response from GREETING_OUTPUT (list of predefined greeting outputs). "query" is tokenized and lemmatized and the result is appended to the list "sent_tokens". sklearn is the library used for TF-IDF vectorization and to calculate the cosine similarity. Cosine similarity is calculated between every sentence from the corpus and the user query, the sentence having the highest cosine similarity is given as the output (cosine similarity values are sorted in descending order and the first value is selected)

4.3.4 Hercules: Using Sequential Modeling

The data is stored in *intents.json* file and it contains a list of intents. Each intent or class has a tag, a pattern, and a response. The "tag" defines the intent or class. The "pattern" is a list of possible questions of the corresponding class. The "response" is a list of possible answers to the questions of that "tag". The chatbot will take the message from the user and identify the "tag" of the message and will give the corresponding response.

Algorithm 12 Algorithm for Chatbot using Tf-idf Vectorization

```
data = load data.txt

sent_tokens = sentence tokenization of data

word_tokens = word tokenization of the text

Remove punctuations from the text

Initialize lists GREETING_INPUT, GREETING_OUTPUT with sample greeting
inputs and outputs

while word  $\in$  sentence.split do
    if word  $\in$  GREETINGS_INPUT then
        Print out a random GREETINGS_OUTPUT
    end if
end while

user_input = Input from the user

user_response = tokenized user query

Append user_response to sent_tokens

tfidfVec = Create tfidf Vector of sent_tokens

vals = Cosine Similarity between "sent_tokens" and "user_response"

idx = Id of the sentence with the highest Cosine Similarity

if req_tfidf = 0 then
    Print "I am sorry, I didnt understand you"
else
    Print sent_tokens[idx]
end if
```

The explanation of Algorithm 13 is given the following paragraphs. Every question of every intent is tokenized using *nltk.word_tokenize()* and is appended to "words" list. All the tags are stores in the "labels" list. Now, "words" is a list that contains all the words in the database. Every word in "words" is converted to lowercase using *lower()* function. Now, all the words in the "words" list are lemmatized using *WordNetLemmatizer().lemmatize()* function and all the duplicate words are removed using *set(words)*. Both "words" and "labels" are sorted. A bag of words is created having variable name "bag", where the size of "bag" is the number of root words in the database. For every word in "words" list, if that word exist in the sentence, then 1 is appended to "bag", else 0 is appended to "bag".

In 2014, authors Nitish Srivastava et al. [31] authors wrote that dropout is a technique that prevents overfitting and provides a way of approximately combining different neural network efficiently. The term "dropout" refers to dropping out units randomly from the hidden or visible layers in the neural network. By dropping a unit, the unit is temporarily removed from the network, along with its incoming and outgoing connections by setting it's weight to zero.

A Neural Network with 3 layers is created. A dense or fully connected input layer of size equal to the size of "bag" having a ReLu activation function. Dropout is used which will drop 50% of the units. A fully connected hidden layer of 64 neurons is created and ReLu activation function is applied to it. Dropout is used which will drop 30% of the units. A dense output layer of size equal to number of tags in the database is created with Softmax activation function. The Softmax activation function converts the output to a list of probabilities, wherein each value denotes the probability of the sentence belonging to the corresponding tag and the sum of all probabilities is equal to 1. The model is optimized using "adam". The Adam Optimizer is an adaptive

learning rate method, which means, it computes individual learning rates for different parameters. When the user query is passed through the neural network, the tag with the highest probability is chosen and the response is given to the user.

4.3.5 ALICE:Using AIML

In AIML, *categories* are the basic unit of knowledge. Each category has a *pattern* and a *template*. The *pattern* describes the query and *template* describes the chatbot's responses. The *template* tag can have a list of possible responses for the chatbot to choose from and it will randomly give one response.

There are two types of AIML classes:

- **Atomic Category:** It is an AIML classification where the query are an exact match. This type of classification does not contain any wildcards.

< category >

< pattern > Good Morning < /pattern >

< template > Good Morning to you too! < /template >

< /category >

- **Default category:** wildcard symbols such as $\wedge$ and $*$ are used in the *pattern*. $*$ wildcard captures one or more words and $\wedge$ wildcard captures 1 or more words.

< category >

< pattern > Hi, $\wedge$ < /pattern >

< template > Hi, Good to see you < /template >

< /category >

Algorithm 13 Algorithm for Chatbot using Sequential Modeling

```
data = load data from JSON file
Implement Lemmatization
Initialize lists words, classes, docx, docy
while intent ∈ data do
    while pattern ∈ intent do
        wrds = tokenize words in the pattern
        Append "wrds" to words
        Append "wrds" to docx
        Append "tag" to docy
    end while
    if tag ∉ labels then
        Append "tag" to labels
    end if
end while
remove punctuations from "words"
words = stemmed "words" converted to lowercase
sort "words" and "labels"
Initialize lists: training, output
while sentence ∈ docsx do
    Initialize "bag" (bag of words)
    wrds = stem every word in "sentence"
    while word ∈ words do
        if word ∈ wrds then
            append 1 to bag
        else
            append 0 to bag
        end if
    end while
    Append "bag" to training

    outputrow[labels.index(docsy[x])] = 1
    Append "bag" to training
    Append outputrow to output
end while
create a Sequential model with softmax activation function
```

AIML consists of aiml objects and each aiml object contains topics and categories. The most important parts of AIML documents are:

- **AIML Root:** This is the root element and it marks the initiation of the aiml document. It is analogous to "html" tag in html documents.

```
< aiml > ..... < /aiml >
```

- **AIML Category:** This is the second level element that contains a single "pattern" and a single "template".

```
< category >
```

```
....Content of the category....
```

```
< /category >
```

- **AIML Pattern:** This tag contains the aiml query or atleast its pattern. It is created using wildcards which are the basis of pattern matching and have priorities assigned to them.

```
< category >
```

```
< pattern >
```

```
....Query....
```

```
< /pattern >
```

```
< /category >
```

- **AIML Template:** It contains the answer to the query in the corresponding category.

```
< category >
```

```
< pattern >
```

```
....Query....
```

```
< /pattern >
```

```
< template >
```

```
....QueryResponse....
```

```
< /template >
```

< /category >

It is possible to program a list of possible responses and let the chatbot choose one answer at random using the "random" and "li" tags.

< category >

< pattern >

....Query....

< /pattern >

< template >

< random >

< li >QueryResponse1.... < /li >

< li >QueryResponse2.... < /li >

< li >QueryResponse3.... < /li >

< /random >

< /template >

< /category >

Algorithm 14 AIML algorithm

chat.py ← file which is a python file.

parse the stdstartup.xml file to it with the appropriate command

Save the data to the brain.dump, file.

stdstartup.xml file is an XML file with the command which will load the aiml files.

create aiml files sing pattern matching

These five chatbots were created to see how different technologies and algorithms have an impact on the performance of a chatbot. Confusion matrices were calculated using the sklearn library. Furthermore, all the five chatbots where implemented and number of epochs were varied. Lastly, 150 queries were implemented on all the chatbots and all the responses were observed.

Chapter 5

Result Analysis

This chapter is divided into three sections. The first section contains the confusion matrices and accuracy which were calculated based on a sample training dataset. In the second section, 150 queries were implemented on all the chatbots and their responses were observed to check if they categorized the query correctly or not. Lastly, the last section contains the screenshots of the conversation with the bots.

5.1 Confusion Matrices and Accuracy based on sample test dataset

A test dataset having 12 queries is used to test the chatbot models created. Confusion matrices and accuracies are calculated using the `sklearn.metrics` library. (using `confusion_matrix` and `accuracy_score` functions)

5.1.1 Smart Bot

In this model, the neural network was created using TensorFlow and multiple pre-processing steps were applied. In the pre-processing phase, Lancaster Stemming algorithm was used which is more accurate. Furthermore, softmax activation function is applied to the output layer which increases the perfor-

	Actual True	Actual False
Predicted True	10	2
Predicted False	2	130

Table 5.1: Confusion matrix for SmartBot

mance of the neural network. Table 5.1 is the confusion matrix of Smart Bot.

5.1.2 Sam

	Actual True	Actual False
Predicted True	8	4
Predicted False	4	128

Table 5.2: Confusion matrix for Sam

In this model, the neural network was created using PyTorch and relu activation function is applied to the input layer and the hidden layers. This impacted the performance of the model. Table 5.2 is the confusion matrix of Sam.

5.1.3 Big Mouth

	Actual True	Actual False
Predicted True	8	4
Predicted False	4	128

Table 5.3: Confusion matrix for Big Mouth

In this model, neural network is not created. Instead TF-IDF Vectorization

is used to convert every sentence into a vector and Cosine Similarity is used to calculate the similarity between every sentence and the query. This model does not understand the meaning of the query, it simply finds the most similar sentence. Table 5.3 is the confusion matrix of Big Mouth.

5.1.4 Hercules

	Actual True	Actual False
Predicted True	10	2
Predicted False	2	130

Table 5.4: Confusion matrix for Hercules

In this model, Sequential modeling is used while creating the neural network which was designed to prevent the problem of overfitting, this in turn improves the performance of the model. Table 5.4 is the confusion matrix of Hercules.

5.1.5 ALICE

	Actual True	Actual False
Predicted True	3	9
Predicted False	9	123

Table 5.5: Confusion matrix for ALICE

In this model, AIML is used to create pattern matching rules. Here, no computation is done and the query is matched to the predefined rules. The programmer needs to have a deep understanding of the AIML functionalities to get acceptable results. Table 5.5 is the confusion matrix of ALICE.

Smart Bot	Sam	Big Mouth	Hercules	ALICE
0.83333	0.66667	0.66667	0.83333	0.25

Table 5.6: Accuracy of each chatbot

Table 5.6 is the comparison of the accuracy of each chatbot. It can be observed that Hercules has the highest accuracy. A possible explanation of this could be that because of sequential modelling the neural network was more adaptable to unknown queries.

5.2 Query Analysis on Chatbots

150 simple queries were created and 15 of them had spelling mistakes in them. All these queries were implemented on all the chatbots and their responses were observed to check if they categorized the query correctly or not implemented.

Table 5.7: Queries.

Begin of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
What is the difference between CCE and IT?	×	✓	✓	✓	✓
How is CCE different from IT?	✓	×	×	✓	×
What is the difference between CCE and CSE?	×	✓	✓	✓	✓
How is CCE different from CSE?	✓	✓	×	✓	×

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
What is CCE at MIT Manipal? What is its scope compared to CSE and IT?	✓	✓	×	✓	×
How is the CCE branch at Manipal?	✓	✓	✓	✓	✓
How is the Computer an communication branch at Manipal?	✓	✓	✓	✓	×
How is the CCE branch?	✓	×	✓	✓	✓
How is the Computer and Communication branch?	✓	×	✓	✓	×
Is the CCE branch good?	✓	×	✓	×	✓
Is the Computer and Communication branch at Manipal good?	✓	✓	✓	×	×
How is the CCE branch at MIT?	×	×	✓	✓	✓
How is the Computer and communication branch at MIT?	×	×	×	✓	×
Is the CCE branch good?	✓	×	✓	×	✓
How is the ECE branch?	✓	✓	×	✓	✓
Is the ECE branch good?	✓	✓	×	✓	✓
How is the ECE branch at Manipal?	✓	✓	×	×	✓

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
How is the ECE branch at MIT?	✓	✓	×	✓	✓
Is ECE branch good?	✓	✓	×	✓	✓
Is the ECE branch bad?	✓	✓	×	✓	✓
How is the Electronics and Communication branch?	×	✓	×	✓	×
Is the Electronics and Communication branch good?	×	✓	×	✓	×
How is the Electronics and Communication branch at Manipal?	×	×	×	✓	×
How is the Electronics and communication branch at MIT?	×	✓	×	✓	×
Is Electronics and Communication branch good?	×	✓	×	✓	×
Is the Electronics and Communication branch bad?	×	✓	×	✓	×
Does the Manipal Institute provide 100% placements?	✓	✓	✓	✓	×
Does the college guarantee 100% placement?	✓	✓	✓	×	×
Is placement guaranteed?	×	×	×	×	×
Is it guaranteed that I will get a job?	×	×	×	×	×

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
Is it guaranteed that I will get placed?	×	×	×	×	×
what are the chances that I will get a job after college?	×	×	×	×	×
How are the placements?	✓	✓	✓	✓	✓
How are the placements for CCE?	✓	✓	✓	✓	×
How are the placements for Computer and Communication branch	✓	✓	×	×	×
Are the placements good?	✓	✓	✓	✓	×
Do cce students get placed in good companies?	✓	×	✓	×	×
what is the placement situation like?	✓	✓	×	✓	×
How are the placements for ECE students?	✓	✓	×	✓	×
How are the placements for ECE students at MIT?	✓	✓	×	✓	×
How are the placements for Electronics and Communication students at Manipal University?	✓	×	×	✓	×
Are the placements of ECE students good?	✓	×	×	✓	×

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
Do ECE students get placed in good companies	✓	×	×	×	×
How are placements for EEE and ECE students at Manipal?	✓	✓	×	✓	×
How are placements for ECE and EEE students at Manipal?	✓	✓	×	✓	×
How are placements for EEE and ECE?	✓	✓	✓	✓	×
How are placements for ECE and EEE?	✓	✓	✓	✓	×
Do ECE students get more job opportunities than EEE students?	×	×	×	×	×
Do companies prefer ECE students over EEE students?	×	×	×	×	×
Do companies prefer EEE students over ECE students?	×	×	×	×	×
Whether separate paper exists for GATE exam or they need to write the paper of Computer Science?	✓	×	✓	✓	✓

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
Is there a separate GATE paper for CCE pass outs?	✓	✓	✓	✓	✓
Is there a seperate paper for cce students?	✓	✓	✓	✓	×
Is there a seprate GATE paper for CCE passouts?	✓	✓	✓	✓	✓
Do CCE students have a seperate GATE paper?	✓	✓	✓	✓	×
What is the procedure for CCE students for appearing in GATE?	×	×	✓	×	×
What is the scope in government sector?	✓	✓	✓	✓	✓
Can CCE students apply for government jobs?	✓	✓	✓	✓	×
Can students apply for government jobs after graduation?	✓	✓	✓	✓	✓
Can CCE passout students apply for government jobs?	✓	✓	✓	✓	×
Can CCE students apply for gvnt jobs?	✓	✓	✓	✓	×
What is the ratio of boys to girls in a class?	✓	✓	✓	✓	✓
What is the ratio of boys to girls?	✓	✓	✓	✓	✓

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
What is the ratio of boys and girls?	✓	✓	✓	✓	✓
What is the ratio of girls to boys in a class?	✓	✓	✓	✓	✓
What is the ratio of girls to boys?	✓	✓	✓	✓	✓
Is entrepreneurship encouraged by the college?	×	✓	×	✓	×
What is the prospect of becoming entrepreneur after completing the course?	✓	✓	×	✓	✓
What are the opportunities for a budding entrepreneur?	✓	✓	×	✓	×
Will the students get help from the college if they want to become an entrepreneur?	✓	✓	×	✓	×
Does college promote entrepreneurship?	×	×	×	✓	×
What is the average package?	✓	✓	✓	×	×
What is the highest package?	×	✓	✓	✓	×
What is the average and the highest package?	×	×	✓	✓	×
What kind of packages do students get?	×	×	×	✓	×

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
What is the range of salaries drawn by the students?	×	×	×	×	×
Do companies prefer CSE students over CCE students?	×	✓	✓	✓	×
Whether the companies which are coming for CSE are coming for CCE?	✓	✓	✓	✓	×
Do companies discriminate against CCE students?	×	×	✓	×	×
Do companies favor CSE students over CCE students?	✓	✓	✓	×	×
What all different specializations are there in CCE?	✓	×	✓	✓	✓
Can CCE students go for specializations?	×	×	×	✓	×
What are different specializations available in CCE?	✓	×	✓	✓	✓
What kind of specializations are available for CCE students?	✓	✓	✓	✓	✓
How many kinds of specializations are available?	✓	×	✓	✓	✓
Are students encouraged to create projects?	✓	✓	✓	✓	×

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
How is the practical / Project exposure in CCE?	✓	✓	✓	✓	×
Do students get mentorship if they want to create projects?	✓	×	✓	✓	×
Are there projects available for students?	✓	✓	✓	✓	✓
Is CCE AICTE approved?	✓	✓	✓	✓	×
Is CCE an AICTE approved branch?	×	✓	✓	✓	×
Is CCE approved by any education program?	×	✓	✓	✓	×
IS CCE course AICTE approved?	✓	✓	✓	✓	×
VIT CSE or manipal CSE I am getting both?	✓	×	✓	×	×
Which college has better CSE program MIT of VIT?	✓	×	×	×	×
Is CSE at VIT better than that in MIT?	×	×	✓	×	×
Is MIT's CSE better than VIT's CSE?	×	×	×	×	×
what to choose between cse at mit and cse at vit?	✓	✓	×	✓	×
How is the IT branch at MIT?	✓	×	✓	×	×

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
Is the IT branch good?	✓	✓	×	✓	×
How is the IT branch at Manipal mian campus?	✓	✓	×	✓	×
How is the IT branch?	✓	✓	×	✓	×
IT or EEE at Manipal Institute of Technology?	✓	✓	×	×	×
Is IT better than EEE?	✓	✓	×	✓	×
Which is better, IT or EEE?	✓	✓	×	×	×
Is EEE better than IT?	✓	✓	×	✓	×
IT or EEE at MIT?	✓	✓	×	×	×
How is the DSE branch at MIT?	✓	✓	×	×	×
Is the DSE branch good?	×	×	×	×	×
How is the Data Science Engineering branch at MIT?	×	×	✓	×	×
Is the Data Science Engineering branch good at Manipal?	×	×	×	×	×
Is IT easier than CSE?	✓	✓	✓	✓	×
Which is easier, IT or CSE?	✓	×	✓	✓	×
EEE or Mechanical?	✓	×	×	✓	×
Which is better, EEE or Mechanical?	✓	✓	×	✓	×
Is EEE better than Mechanical?	✓	✓	×	×	×

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
Is Mechanical better than EEE at MIT?	✓	✓	✓	×	×
Should I take a drop year?	✓	✓	✓	✓	×
What are your thoughts on drop year?	✓	×	✓	✓	×
Should I take EEE/ECE in MIT Manipal or drop a year?	✓	✓	✓	✓	×
What are the benefits of a drop year?	✓	×	✓	×	×
What is the difference between CCE and IT?	✓	×	×	✓	×
How is CCE different from CSE?	✓	×	×	✓	×
How is the CCE branch at Manipal?	✓	✓	×	×	✓
Is the CCE branch good?	✓	×	×	×	✓
Does MIT guarantee hundred percent placement?	✓	×	✓	✓	×
How are the placements for CCE?	×	✓	×	×	×
How are the placements for ECE students?	×	✓	✓	✓	✓
How are the placements for ECE and EE students?	×	×	×	×	✓

Continuation of Table					
Query	Smart Bot	Sam	Big Mouth	Hercules	ALICE
Can CCE students apply abroad for masters?	×	×	×	×	×
Is there a separate paper for GATE for CCE students?	✓	✓	✓	✓	✓
Can CCE students apply for government jobs?	✓	✓	✓	✓	×
What is the ratio of boys and girls?	✓	✓	✓	✓	×
What is the average and highest package?	✓	✓	×	✓	×
What all specializations are available in CCE?	×	×	✓	✓	×
Is the course AISTE approved?	✓	✓	✓	✓	×
Should I take a drop year?	✓	✓	✓	✓	×
Is CCE AISTE approved?	✓	✓	✓	✓	×
What is the scope in government sector?	✓	✓	✓	✓	×
Does the Manipal Institute provide 100 percent placements?	✓	✓	✓	✓	×
End of Table					

From Table 5.7, the number of queries which were correctly answered by each chatbot can be observed. Table 5.8 depicts how many queries were correctly

answered and the accuracy of each model.

Table 5.8: percentages

Chatbot	Number of Queries Correctly Identified	Accuracy
SmartBot	108	72
Sam	95	63.3333
BigMouth	84	56
Hercules	110	73.3333
Alice	36	24

5.3 Conversational Analysis of Chatbots

In this section simple and compound queries are implemented on all the chatbots and the conversation is shown in the form of screenshots. For Smart Bot, Sam and Hercules; training loss is calculated and its variations are noted when number of epochs is changes.

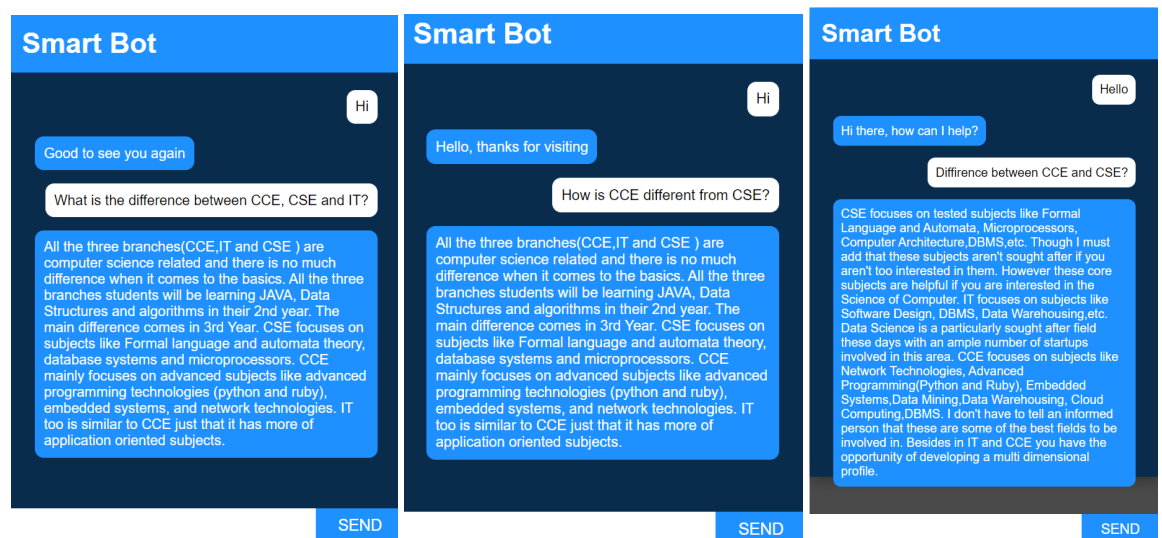
5.3.1 Smart Bot

```
--
Training Step: 4996 | total loss: +[1m+32m0.53522+0m+0m | time: 0.001s
| Adam | epoch: 1000 | loss: 0.53522 - acc: 0.9601 -- iter: 08/40
+--+
Training Step: 4997 | total loss: +[1m+32m0.48328+0m+0m | time: 0.002s
| Adam | epoch: 1000 | loss: 0.48328 - acc: 0.9641 -- iter: 16/40
+--+
Training Step: 4998 | total loss: +[1m+32m0.43615+0m+0m | time: 0.004s
| Adam | epoch: 1000 | loss: 0.43615 - acc: 0.9677 -- iter: 24/40
+--+
Training Step: 4999 | total loss: +[1m+32m0.39375+0m+0m | time: 0.006s
| Adam | epoch: 1000 | loss: 0.39375 - acc: 0.9709 -- iter: 32/40
+--+
Training Step: 5000 | total loss: +[1m+32m0.35567+0m+0m | time: 0.007s
| Adam | epoch: 1000 | loss: 0.35567 - acc: 0.9738 -- iter: 40/40
--
```

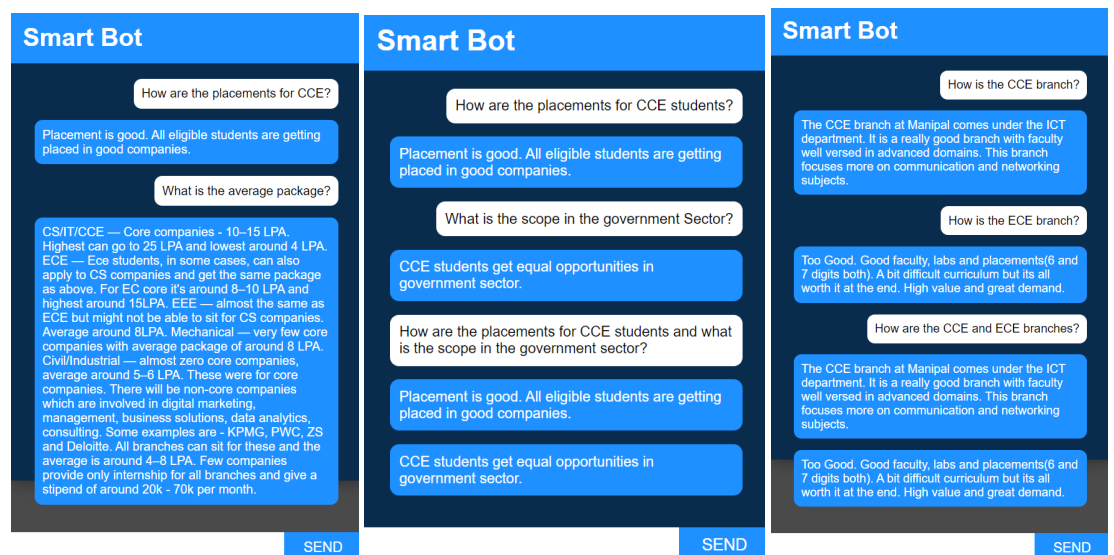
At 1000th epoch, the training loss of the model is 0.35567 and the accuracy is 0.9738. If we increase the number of epochs to 1500, the training loss of the model reduces to 0.15079 and the accuracy increases to 0.9949. On the contrary, if number of epochs becomes 500, the training loss becomes 0.24558 and the accuracy becomes 0.9817.

It can be observed that increasing the number of epochs increases the accuracy

of the chatbot and decreases the training loss.



In the above figures, it can be seen the chatbot gives the correct output for the queries for which it was not trained. For example, the first query is in the dataset whereas the second query is not in the dataset. Furthermore, even if there is the spelling mistake in the query asked by the user, the model gives the correct output.



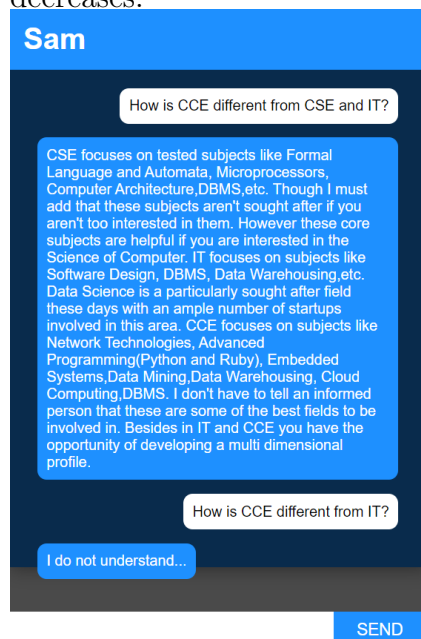
For complex and Compound queries, the chatbot gives accurate results. It can be seen in the above figures that the chatbot gives the answers to both the queries.

5.3.2 Sam

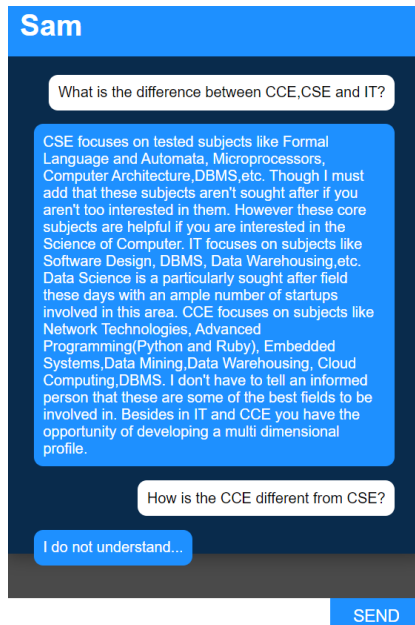
```
Epoch [100/1000], Loss: 2.2308
Epoch [200/1000], Loss: 0.0224
Epoch [300/1000], Loss: 0.0068
Epoch [400/1000], Loss: 0.0032
Epoch [500/1000], Loss: 0.0024
Epoch [600/1000], Loss: 0.0047
Epoch [700/1000], Loss: 0.0081
Epoch [800/1000], Loss: 0.0002
Epoch [900/1000], Loss: 0.0017
Epoch [1000/1000], Loss: 0.0003
final loss: 0.0003
training complete. file saved to data.pth
```

At the 1000th epoch, the training loss of the chatbot is 0.0003. If the number of epochs is increased to 1500, the loss in training decreases to 0.0001. On the contrary, if the number of epochs is decreased to 500, then the training loss increases to 0.0017.

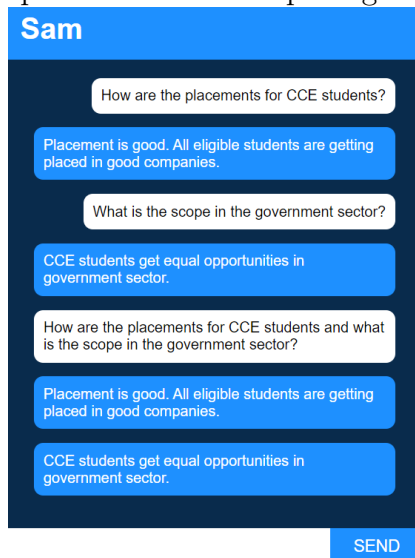
It can be observed that increasing the number of epochs, the training loss decreases.



In the above figure, the model gives the correct answers to the queries which are in the dataset but gives the wrong answers to the queries which are not in the dataset. One of the possible explanations of this is overfitting of the model on the training dataset. Overfitting makes the model relevant only to the dataset on which it was trained and irrelevant to all the other datasets.

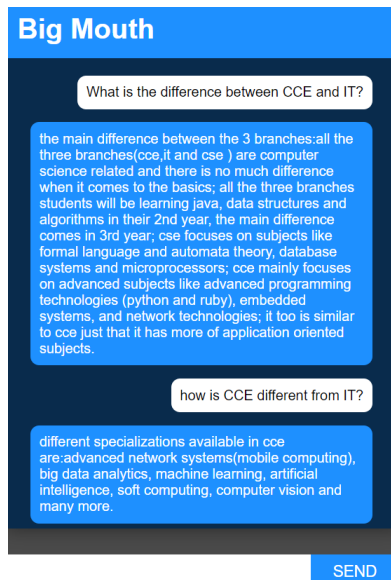


In the above figure, it is observed that the bot does not give the correct response if there is a spelling mistake in the query.

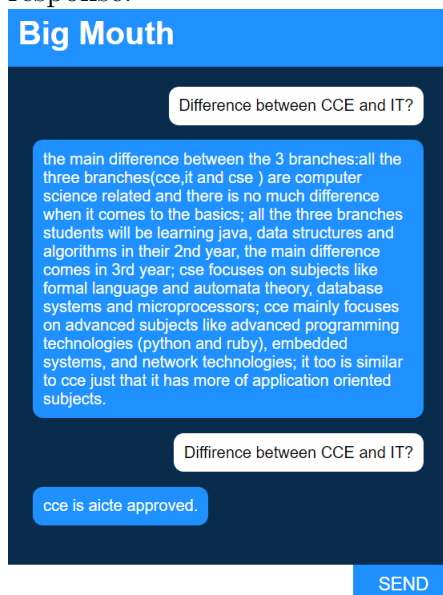


For complex and Compound queries, the chatbot gives accurate results. It can be seen in the above figures that the chatbot gives the answers to both the queries.

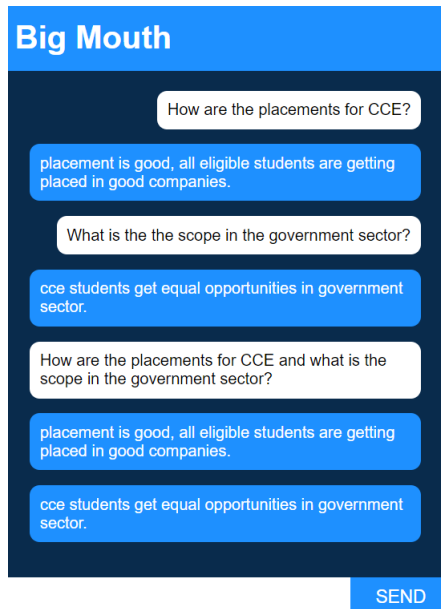
5.3.3 Big Mouth



In the above figure, the model gives the correct answers to the direct queries, but if the queries are asked in a different manner, it may not give the correct response.



In the above figure, it is observed that the bot might not give the correct response if there is a spelling mistake in the query.



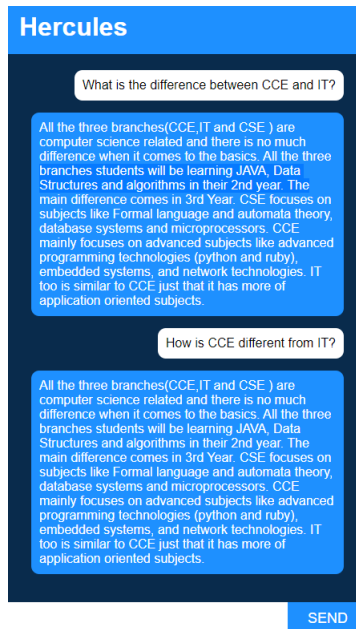
For complex and Compound queries, the chatbot gives accurate results. It can be seen in the above figures that the chatbot provides answers to both the queries.

5.3.4 Hercules

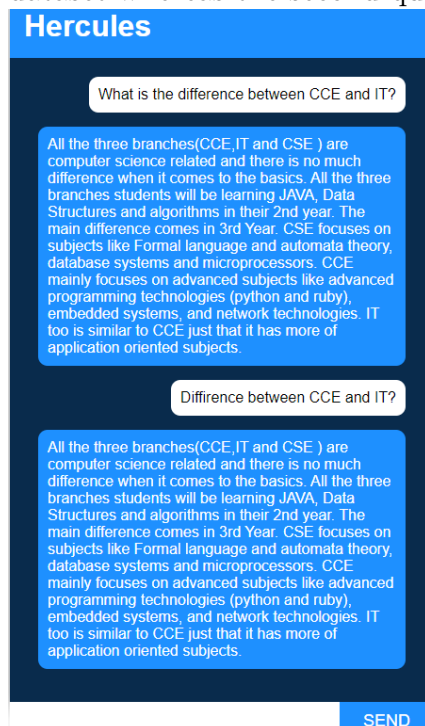
```
Epoch 198/200
2/2 [=====] - 0s 2ms/step - loss: 2.3981e-04 - accuracy: 1.0000
Epoch 199/200
2/2 [=====] - 0s 2ms/step - loss: 0.0076 - accuracy: 1.0000
Epoch 200/200
2/2 [=====] - 0s 3ms/step - loss: 0.0354 - accuracy: 0.9831
```

At 200th epoch, the training loss of the model is 0.0354 and the accuracy is 0.9831. If we increase the number of epochs to 250, the training loss of the model increases to 0.2196 and the accuracy reduces to 0.9800. On the contrary, if number of epochs becomes 150, the training loss becomes 0.1089 and the accuracy becomes 0.9400.

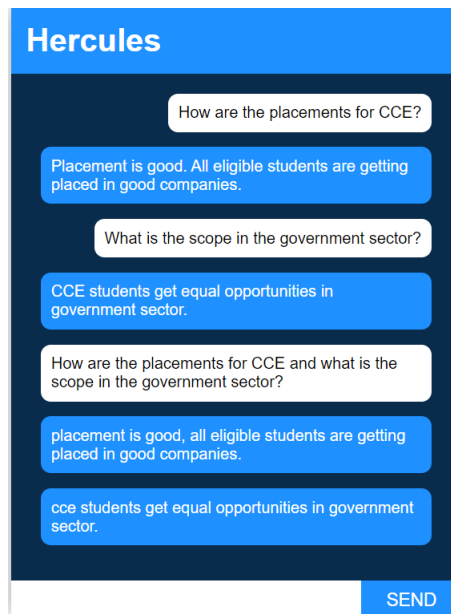
It can be observed that increasing the number of epochs increases the accuracy of the chatbot and decreases the training loss. Similar trend is seen if the number of epochs is decreased.



In the above picture, it can be seen the chatbot gives the correct output for the queries for which it was not trained. For example, the first query is in the dataset whereas the second query is not in the dataset.

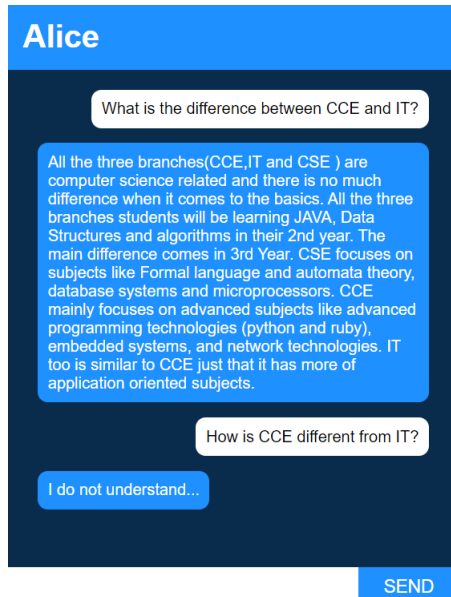


In the above figure, it is observed that the bot can give the correct response even if there is a spelling mistake in the query.

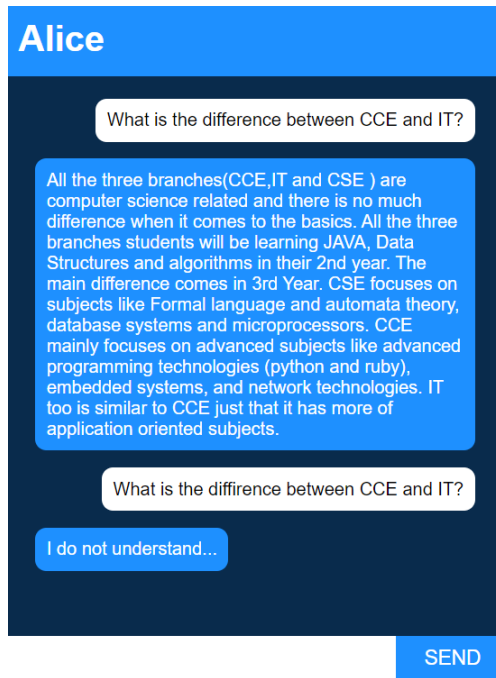


For complex and Compound queries, the chatbot gives accurate results. It can be seen in the above figures that the chat provides answers to both the halves of the query.

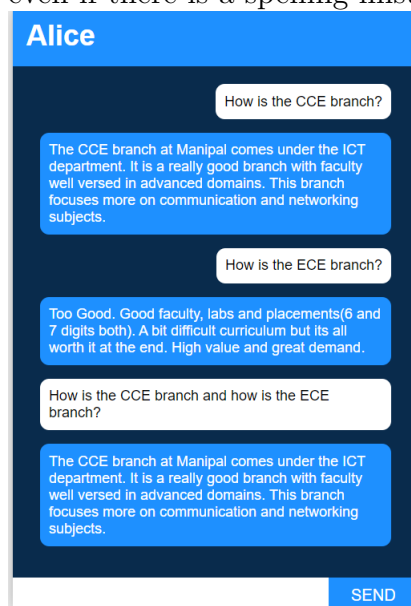
5.3.5 ALICE



In the above picture, it can be seen the chatbot gives the correct output for the queries for which it was not trained. For example, the first query is in the dataset whereas the second query is not in the dataset.



In the above figure, it is observed that the bot does n give the correct response even if there is a spelling mistake in the query.



For complex and Compound queries, the chatbot does not give accurate results. It can be seen in the above figures that the answer provided by the chatbot is incomplete as it only answers one query.

Considering Sam as the base chatbot, it can be observed that Smart Bot and Hercules have better performance than Sam. Furthermore, it can be observed that Big Mouth and Alice do not perform well as compared to Sam.

SmartBot, Sam and Hercules are all neural network-based models and their percentage of "yes" is greater than 60 percent. If these three chatbots are arranged in the increasing order of percentage of "yes", then the order will be Sam<Smart bot<Hercules. Since all the three models are neural network-related, it can be concluded that:

- Type of stemming algorithms applied to the data might affect the performance of the neural network. In Smart Bot LancasterStemmer algorithm was used, whereas, in Sam PorterStemmer algorithm was used.
- The type of activation function and the layers on which it is applied might affect the performance of the neural network. In Smart Bot, softmax activation function is applied to the output layer and no activation function is applied to any other layer. On the other hand, in Sam, relu activation function is applied to each node in the input layer and every node in the hidden layers as well and no activation function is applied to the output layer.
- It can be noticed that Hercules has a Sequential Neural Network which is designed to prevent overfitting. This may be the reason that Hercules has the highest percentage of "yes".
- It can be observed that Hercules is the only chatbot which has any kind of optimizer applied, which improves its performance.

Considering Sam as the base chatbot, it can be observed that Alice (pattern matching) and Big Mouth (TF-IDF vectorization) might not be as effective as neural network-based models and they require a deeper understanding of the technologies to obtain an acceptable model.

Chapter 6

Conclusion and Future Scope

Engineering colleges follow an online admission process which involves a counseling process for engineering stream selection. During the counseling phase, students and parents have a lot of queries regarding the branches offered by the college and many other such queries. These questions can be answered by visiting the college or over a phone call. The volume of the queries can be overwhelming and due to this there might be some miscommunication of information; or many a times, the officials might be busy on other calls. Students might have to rely on unofficial sources of information like Quora to get information. Furthermore, the students have to navigate through the entire website for information which can be tedious.

In this project, five chatbot models were created using neural networks, TF-IDF vectorization, and pattern matching. In neural network-related models, pre-processing steps like converting to lowercase, stemming, lemmatization, tokenization, removal of stop words and creation of "bag of words" are applied to the training data before passing it through the neural network. A query is taken from the user; pre-processing steps are applied to it, and it is passed through the model which returns the list of probabilities that the query belongs to a certain intent.

In the TF-IDF model, the dataset is split into sentences, pre-processed and

is converted to vectors. A query is taken as input and it is also converted to a vector. Now, Cosine Similarity is calculated between every sentence vector in the dataset and the query vector. The sentence having the highest cosine similarity is returned as output.

In the pattern matching approach, Artificial Intelligence Markup Language (AIML) is used to create heuristic rules for pattern matching using wildcards and AIML objects. The bot is also programmed to understand the user's name and remember it throughout the conversation. A query is taken from the user and the chatbot tries to identify the pattern and returns the corresponding response.

Among the five chatbots discussed in the project, Hercules has the best performance because it has sequential modeling which is designed to prevent overfitting of training data. Furthermore, it is the only chatbot which has any kind of optimizer applied to it which improves its performance. Therefore, it can be concluded that chatbot similar to Hercules can be implemented in real time for MIT counselling process. This will be very helpful for the students because it can provide official and accurate results. Furthermore, it can provide 24x7 assistance and the information provided will be uniform. Lastly, students will not have to depend on unreliable sources of information for resolving their queries.

In the future, the following elements can be added:

- Increase the training dataset and include more information. .
- Voice queries can be implemented.

References

- [1] T. Lalwani, S. Bhalotia, A. Pal, V. Rathod, and S. Bisen, “Implementation of a chatbot system using ai and nlp,” *International Journal of Innovative Research in Computer Science & Technology (IJIRCST) Volume-6, Issue-3*, 2018.
- [2] J. Thukrul, A. Srivastava, and G. Thakkar, “Doctorbot-an informative and interactive chatbot for covid-19,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 07, pp. 2395–0072, 2020.
- [3] S. Maher, S. Kayte, and S. Nimbhore, “Chatbots & its techniques using ai: an review,” *International Journal for Research in Applied Science and Engineering Technology*, vol. 8, no. 12, pp. 503–508, 2020.
- [4] M. Aleedy, H. Shaiba, and M. Bezbradica, “Generating and analyzing chatbot responses using natural language processing,” *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 9, 2019.
- [5] P. Qi, Y. Zhang, Y. Zhang, J. Bolton, and C. D. Manning, “Stanza: A python natural language processing toolkit for many human languages,” *arXiv preprint arXiv:2003.07082*, 2020.
- [6] M. M. H. Dihyat and J. Hough, “Can rule-based chatbots outperform neural models without pre-training in small data situations?: A preliminary comparison of aiml and seq2seq,” in *Proceedings of the 25th Workshop on the Semantics and Pragmatics of Dialogue*, 2021.

- [7] A. Chandan, M. Chattopadhyay, and S. Sahoo, "Implementing chat-bot in educational institutes," *IJRAR Journal*, vol. 6, no. 2, pp. 44–47, 2019.
- [8] V. Sharma, M. Goyal, and D. Malik, "An intelligent behaviour shown by chatbot system," *International Journal of New Technology and Research*, vol. 3, no. 4, p. 263312, 2017.
- [9] E. Adamopoulou and L. Moussiades, "Chatbots: History, technology, and applications," *Machine Learning with Applications*, vol. 2, p. 100006, 2020.
- [10] C. Curry and J. D. O'Shea, "The implementation of a story telling chat-bot," *Advances in Smart Systems Research*, vol. 1, no. 1, p. 45, 2012.
- [11] L. Fryer and R. Carpenter, "Bots as language learning tools," *Language Learning & Technology*, vol. 10, no. 3, pp. 8–14, 2006.
- [12] S. Verma, L. Sahni, and M. Sharma, "Comparative analysis of chatbots," in *Proceedings of the International Conference on Innovative Computing & Communications (ICICC)*, 2020.
- [13] A. A. Qaffas, "Improvement of chatbots semantics using wit. ai and word sequence kernel: Education chatbot as a case study," *International Journal of Modern Education and Computer Science*, vol. 11, no. 3, p. 16, 2019.
- [14] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," *arXiv preprint arXiv:1603.04467*, 2016.
- [15] S. Qaiser and R. Ali, "Text mining: use of tf-idf to examine the relevance of words to documents," *International Journal of Computer Applications*, vol. 181, no. 1, pp. 25–29, 2018.

- [16] I. Ahmed and S. Singh, “Aiml based voice enabled artificial intelligent chatterbot,” *International Journal of u-and e-Service, Science and Technology*, vol. 8, no. 2, pp. 375–384, 2015.
- [17] R. Rani and D. Lobiyal, “Automatic construction of generic stop words list for hindi text,” *Procedia computer science*, vol. 132, pp. 362–370, 2018.
- [18] D. Ofer, N. Brandes, and M. Linial, “The language of proteins: Nlp, machine learning & protein sequences,” *Computational and Structural Biotechnology Journal*, vol. 19, pp. 1750–1758, 2021.
- [19] T. Ahmad and M. N. Aziz, “Data preprocessing and feature selection for machine learning intrusion detection systems,” *ICIC Express Lett*, vol. 13, no. 2, pp. 93–101, 2019.
- [20] B. Vijayakumar and M. M. M. Fuad, “A new method to identify short-text authors using combinations of machine learning and natural language processing techniques,” *Procedia Computer Science*, vol. 159, pp. 428–436, 2019.
- [21] T. Verma, R. Renu, and D. Gaur, “Tokenization and filtering process in rapidminer,” *International Journal of Applied Information Systems*, vol. 7, no. 2, pp. 16–18, 2014.
- [22] J. Kaur and P. K. Buttar, “A systematic review on stopword removal algorithms,” *International Journal on Future Revolution in Computer Science & Communication Engineering*, vol. 4, no. 4, pp. 207–210, 2018.
- [23] A. I. Kadhim, “An evaluation of preprocessing techniques for text classification,” *International Journal of Computer Science and Information Security (IJCSIS)*, vol. 16, no. 6, pp. 22–32, 2018.

- [24] V. Srividhya and R. Anitha, “Evaluating preprocessing techniques in text categorization,” *International journal of computer science and application*, vol. 47, no. 11, pp. 49–51, 2010.
- [25] D. Khyani, B. Siddhartha, N. Niveditha, and B. Divya, “An interpretation of lemmatization and stemming in natural language processing,” *Shanghai Ligong Daxue Xuebao/Journal of University of Shanghai for Science and Technology*, vol. 22, pp. 350–357, 2020.
- [26] A. Trivedi, V. Gor, and Z. Thakkar, “Chatbot generation and integration: A review,” *International Journal of Advance Research, Ideas and Innovations in Technology*, vol. 5, no. 2, pp. 1308–1311, 2019.
- [27] Z. C. Lipton, J. Berkowitz, and C. Elkan, “A critical review of recurrent neural networks for sequence learning,” *arXiv preprint arXiv:1506.00019*, 2015.
- [28] R. B. Arif, M. A. B. Siddique, M. M. R. Khan, and M. R. Oishe, “Study and observation of the variations of accuracies for handwritten digits recognition with various hidden layers and epochs using convolutional neural network,” in *2018 4th International Conference on Electrical Engineering and Information & Communication Technology (iCEEiCT)*. IEEE, 2018, pp. 112–117.
- [29] S. Sinha, T. Singh, V. Singh, and A. Verma, “Epoch determination for neural network by self-organized map (som),” *Computational Geosciences*, vol. 14, no. 1, pp. 199–206, 2010.
- [30] A. Jaglan, D. Trehan, U. Megha, and P. Singhal, “Covid-19 trend analysis using machine learning techniques,” *Int J Sci Eng Res*, vol. 11, no. 12, pp. 1162–1167, 2020.

- [31] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: <http://jmlr.org/papers/v15/srivastava14a.html>

Table 6.1: Project Detail

Student Details

Student Name	Ankit Agrawal		
Registration Number	180953278	Section/Roll No.	A/48
Email Address	nishu.aa@gmail.com	Phone No.(M)	9711664694

Project Details

Project Title	Title of your project		
Project Duration	4-6 Months	Date of Reporting	01-02-2022

Internal Guide Details

Faculty Name	Dr.Girija Attigeri		
Full Contact Address with PIN Code	Department of Information and Communication Technology, Manipal Institute of Technology, Manipal-576104		
Email Address	girija.attigeri@manipal.edu		

INSPECTOR

ORIGINALITY REPORT

8%

SIMILARITY INDEX

7%

INTERNET SOURCES

7%

PUBLICATIONS

1%

STUDENT PAPERS

PRIMARY SOURCES

1

hectordelgado.me

Internet Source

2%

2

hal.inria.fr

Internet Source

1%

3

www.acmweb.org

Internet Source

1%

4

Shankar Bhausabheb Nikam. "Wavelet energy signature and GLCM features-based fingerprint anti-spoofing", 2008 International Conference on Wavelet Analysis and Pattern Recognition, 08/2008

Publication

1%

5

Jinesh Mehta, Eshaan Ramnani, Sanjay Singh. "Face Detection and Tagging Using Deep Learning", 2018 International Conference on Computer, Communication, and Signal Processing (ICCCSP), 2018

Publication

1%

6

Internet Research, Volume 23, Issue 2 (2013-05-27)

1%



Offer: Computer Consultancy
Ref: TCSL/DT20218056276/Bangalore
Date: 01/09/2021

Mr. Ankit Agrawal
H-16, First Floor, South City 2, Sohna Road, Sohna Road,
Narayana E Techno School,
Gurugram-122018,
Haryana.
Tel# 91-9354372231

Dear Ankit Agrawal,

Sub: Letter of Offer

Thank you for exploring career opportunities with TATA Consultancy Services Limited (TCSL). You have successfully completed our initial selection process for 'TCS Digital' and we are pleased to make you an offer of employment.

You have been selected for the position of **Systems Engineer** in Grade C1. You will be assigned a challenging role in any Business Unit as per the business requirements of TCSL.

Your gross salary including all benefits and Retention Incentive will be INR [REDACTED] per annum, as per the terms and conditions set out herein. Over and above this, you will also be eligible for Learning Incentives (Readiness Incentive and/or Competency Incentive) basis your performance in TCS Xplore Program which gives you an additional earning potential of upto [REDACTED] during the first year. Annexure-1 provides the break-up of the compensation package.

Kindly confirm your acceptance of this offer online through the option 'Accept Offer letter'. If not accepted within 7 days, it will be construed that you are not interested in this employment and this offer will be automatically withdrawn.

After you accept this offer, you will be given a joining letter indicating the details of your joining date and initial place of posting. The Joining letter will be issued to you only upon successful completion of your academic course, you meeting the TCS eligibility criteria & you completing the mandatory pre-joining learning curriculum named TCS Xplore (detailed under Terms & Conditions). You will also be issued a letter of appointment at the time of your joining after

TCS Confidential
TCSL/DT20218056276

1

TATA CONSULTANCY SERVICES

VYDEHI RC-1 BLOCK, 82, EPIP, Whitefield, Bangalore 560 066 India
Tel: 91 80 6724 7000 Fax: 91 80 2841 0114 Website: www.tcs.com
Registered Office Nirmal Building, 9th Floor, Nariman Point, Mumbai 400 021
TCS Careers Serviceline: 1800 209 3111 Email: careers@tcs.com